

UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN



**SISTEMA DE CONTROL Y SEGUIMIENTO DE TRÁMITE DE CERTIFICADOS DE
TALLERES TÉCNICOS EN LA ESCUELA PROFESIONAL DE INGENIERÍA DE
SISTEMAS Y COMPUTACIÓN**

Documento Técnico de Implementación

DOCENTE:

Mg. Raúl Fernández Bejarano

ESTUDIANTES:

SANTILLAN TRUJILLO, Brandon Kody
CORONEL GUEVARA, Jack Yhems

CICLO ACADÉMICO:

VIII

HUANCAYO – PERÚ

Arquitectura de Software – Sección B1 · 2026

Sistema EPISC de Certificados de Talleres Técnicos

Documento técnico de implementación, base de datos, despliegue y verificación.

Dato	Descripción
Proyecto	proyecto-episc-certificados
Arquitectura	Spring Boot + microservicios + API Gateway + MySQL + Docker Compose
Frontend publicado	https://episc-certificados.web.app
Backend local	http://localhost:8080 por API Gateway
Fecha	14/07/2026

1. Resumen Ejecutivo

Se implementó un sistema académico para el control y seguimiento de certificados de talleres técnicos EPISC. El proyecto fue organizado como una arquitectura de microservicios en Spring Boot, usando Docker Compose para levantar los servicios, MySQL como motor de base de datos y Firebase Hosting para publicar el frontend estático.

Para una entrega universitaria se usó una sola instancia MySQL con schemas independientes por microservicio. Esta decisión mantiene separación lógica de datos, reduce complejidad operativa y permite demostrar los principios de independencia de datos sin crear múltiples servidores de base de datos.

2. Trabajo Realizado

- Se revisó el frontend original y se integró dentro del API Gateway de Spring Boot.
- Se corrigieron textos con problemas de codificación, por ejemplo, Técnicos, Computación, Contraseña y Código.
- Se configuraron microservicios Spring Boot para autenticación, solicitudes, trámites, certificados y reportes.
- Se agregó MySQL 8.4 en Docker Compose con schemas separados.
- Se agregaron usuarios demo y datos iniciales de talleres.
- Se verificó Docker Desktop, WSL2 y la ejecución de contenedores.
- Se publicó el frontend en Firebase Hosting.
- Se configuró Firebase para redirigir la raíz del sitio hacia login.html.

3. Arquitectura General

```
Frontend HTML/CSS/JS
|
v
Firebase Hosting o API Gateway estático
|
v
api-gateway :8080
|
+-- ms-auth           :8085 -> db_auth
+-- ms-solicitudes    :8081 -> db_solicitudes
+-- ms-tramites       :8082 -> db_tramites
+-- ms-certificados   :8083 -> db_certificados
+-- ms-reportes       :8084 -> consume APIs internas
|
v
MySQL 8.4 en Docker
```

Componente	Puerto	Responsabilidad	Base/schema
api-gateway	8080	Punto único de entrada y frontend integrado	–
ms-auth	8085	Login, usuarios y JWT	db_auth
ms-solicitudes	8081	Registro de solicitudes y talleres	db_solicitudes
ms-tramites	8082	Avance de etapas e historial	db_tramites
ms-certificados	8083	Emisión y descarga de certificados	db_certificados
ms-reportes	8084	Reportes consultando servicios internos	–
episc-mysql	3306	Motor MySQL 8.4	Schemas separados

4. Base de Datos

La base de datos se ejecuta en el contenedor episc-mysql. El usuario configurado para la demo es root/root. Cada microservicio tiene su propio schema, lo que simula independencia de datos.

Schema	Microservicio	Tablas principales
db_auth	ms-auth	usuario
db_solicitudes	ms-solicitudes	solicitud, taller
db_tramites	ms-tramites	historial_auditoria
db_certificados	ms-certificados	certificado

5. Comandos Para Ejecutar el Proyecto

Abrir PowerShell en la raíz del proyecto:

```
cd "C:\Users\diego\Music\proyecto-episc-certificados\proyecto-episc-certificados"
```

Levantar todos los servicios:

```
docker compose up --build
```

Levantar en segundo plano:

```
docker compose up --build -d
```

Verificar contenedores:

```
docker ps
```

Detener contenedores sin borrar datos:

```
docker compose down
```

6. Comandos Para Verificar MySQL

Entrar al contenedor de MySQL:

```
docker exec -it episc-mysql mysql -u root -p
```

Password:

```
root
```

Ver schemas disponibles:

```
SHOW DATABASES;
```

Ver todas las tablas importantes y sus columnas:

```
USE db_auth;
SHOW TABLES;
DESCRIBE usuario;
SELECT id_usuario, nombre_completo, correo, rol, activo FROM usuario;

USE db_solicitudes;
SHOW TABLES;
DESCRIBE taller;
DESCRIBE solicitud;
SELECT id_taller, nombre_taller, docente_responsable, horas_academicas, activo
FROM taller;
SELECT * FROM solicitud;

USE db_tramites;
SHOW TABLES;
DESCRIBE historial_auditoria;
SELECT * FROM historial_auditoria;

USE db_certificados;
SHOW TABLES;
DESCRIBE certificado;
SELECT * FROM certificado;
```

Consulta rápida para contar registros por tabla:

```
SELECT 'db_auth.usuario' AS tabla, COUNT(*) AS registros FROM
db_auth.usuario
UNION ALL
SELECT 'db_solicitudes.taller', COUNT(*) FROM db_solicitudes.taller
UNION ALL
SELECT 'db_solicitudes.solicitud', COUNT(*) FROM
db_solicitudes.solicitud
UNION ALL
SELECT 'db_tramites.historial_auditoria', COUNT(*) FROM
db_tramites.historial_auditoria
UNION ALL
SELECT 'db_certificados.certificado', COUNT(*) FROM
db_certificados.certificado;
```

7. Endpoints Principales

Servicio	Método	Endpoint	Uso
Auth	POST	/api/v1/auth/login	Iniciar sesión y obtener token JWT
Auth	POST	/api/v1/auth/usuarios	Crear usuarios
Talleres	GET	/api/v1/talleres	Listar talleres
Talleres	POST	/api/v1/talleres	Crear taller
Solicitudes	POST	/api/v1/solicitudes	Registrar solicitud
Solicitudes	POST	/api/v1/solicitudes/{id}/informe	Subir informe PDF
Solicitudes	GET	/api/v1/solicitudes/código/{códigoTramite}	Consultar por código
Solicitudes	GET	/api/v1/solicitudes	Listar solicitudes
Trámites	POST	/api/v1/tramites/avanzar-etapa	Avanzar flujo del trámite
Trámites	GET	/api/v1/tramites/historial/{códigoTramite}	Ver historial de un trámite
Certificados	POST	/api/v1/certificados/emitir	Emitir certificado
Certificados	GET	/api/v1/certificados/{códigoTramite}/descargar	Descargar certificado
Reportes	GET	/api/v1/reportes	Ver reportes

8. Pruebas Rápidas de API

Probar login desde PowerShell:

```
$body = @{ correo = "alumno@gmail.com"; password = "123456" } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://localhost:8080/api/v1/auth/login \
-ContentType "application/json" -Body $body
```

Listar talleres:

```
Invoke-RestMethod http://localhost:8080/api/v1/talleres
```

Listar solicitudes:

```
Invoke-RestMethod http://localhost:8080/api/v1/solicitudes
```

9. Usuarios Demo

Rol	Correo	Contraseña
Alumno	alumno@gmail.com	123456
Coordinador	coordinador@gmail.com	123456
Secretaria	secretaria@gmail.com	123456

10. Publicación en Firebase Hosting

Firebase Hosting se usó para publicar el frontend estático. La carpeta publicada fue api-gateway/src/main/resources/static. La raíz del sitio fue configurada para redirigir a login.html.

```
firebase.cmd login
firebase.cmd init hosting
firebase.cmd deploy --only hosting
```

Configuración final de Firebase (firebase.json):

```
{
  "hosting": {
    "public": "api-gateway/src/main/resources/static",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "redirects": [
      {
        "source": "/",
        "destination": "/login.html",
        "type": 302
      }
    ]
  }
}
```

Nota importante: Firebase Hosting no ejecuta los microservicios Spring Boot. Para que el login funcione desde la URL publicada, el backend debe estar disponible. En la demo local se usó `http://localhost:8080` mientras Docker está encendido en la PC.

11. Checklist de Evidencias para Sustentación

- Captura de docker ps mostrando api-gateway, ms-auth, ms-solicitudes, ms-tramites, ms-certificados, ms-reportes y episc-mysql.
- Captura de login en Firebase Hosting.
- Captura de login exitoso con alumno@gmail.com.
- Captura del panel del alumno.
- Captura de MySQL con SHOW DATABASES.
- Captura de SELECT en db_auth.usuario.
- Captura de SELECT en db_solicitudes.taller.
- Captura de la URL publicada en Firebase.

12. Explicación Breve para Exponer

El proyecto aplica arquitectura de microservicios con Spring Boot. El API Gateway centraliza las entradas y enruta las solicitudes hacia servicios especializados. Cada microservicio tiene responsabilidad propia y datos separados por schema en MySQL. Docker Compose permite levantar toda la solución de manera reproducible. El frontend fue publicado en Firebase Hosting para mostrar una URL pública, mientras que el backend queda dockerizado y listo para ser desplegado posteriormente en Cloud Run, Render, Railway u otra plataforma compatible con contenedores.

13. Sigüientes Mejoras Recomendadas

- Publicar backend en Cloud Run, Render o Railway para que el sistema funcione desde cualquier computadora.
- Cambiar JWT_SECRET y credenciales de base de datos antes de producción.
- Separar bases de datos físicas si el proyecto deja de ser académico y pasa a producción.
- Agregar validaciones y pruebas automatizadas por microservicio.
- Agregar documentación Swagger/OpenAPI para probar endpoints desde navegador.